TRISHA ACADEMY

DevOps Career Track · Student Notes





Kubernetes Class 7

Storage and StatefulSets

Student notes - concepts, diagrams, live lab and 20 interview questions

60-90 minutes **Everyday examples** **Diagrams** **Live lab** **20 interview Q&A**

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

Class 7: Storage, PersistentVolumes and StatefulSets 3

7.1 The disappearing data problem 3

7.2 PersistentVolume and PersistentVolumeClaim 4

7.3 StorageClass and dynamic provisioning 5

7.4 StatefulSets 5

7.5 Live lab 7

Class 7 - Top 20 interview questions 8

K8S · CLASS 07 · 60-90 minutes

Storage, PersistentVolumes and StatefulSets

By the end you can give pods durable storage, explain PV/PVC/StorageClass, and run a stateful app whose pods keep their names and their data.

What you will learn today

- Why container storage disappears
- Volume types: emptyDir, hostPath, configMap/secret, CSI
- PersistentVolume, PersistentVolumeClaim and binding
- StorageClass and dynamic provisioning
- Access modes, reclaim policies, volume expansion and snapshots
- StatefulSets: stable names, ordered startup, per-pod storage
- Lab: a StatefulSet that keeps its data after pod deletion

Part	Topic	Time
1	Story and volumes	10 min
2	PV, PVC, StorageClass	20 min
3	StatefulSets	15 min
4	Live lab	25 min
5	Recap and interview questions	10 min

7.1 The disappearing data problem

Ravi's story
Ravi ran MySQL as a Deployment. A node was patched and rebooted on Sunday, the MySQL pod was recreated on another node... with an empty database. All test orders were gone. Container file systems are temporary; the data had lived and died with the container.

A container's writable layer is deleted when the container is. Kubernetes **volumes** attach storage to a pod, and **persistent** volumes keep data beyond the life of any pod.

Volume type	Lifetime	Use
emptyDir	As long as the pod (survives container restarts)	Scratch space, cache, sharing files between containers
hostPath	As long as the node	Node agents reading /var/log; avoid for apps - ties pod to a node and is a security risk
configMap / secret / downwardAPI	Managed by Kubernetes	Config files (Class 6)
persistentVolumeClaim	Independent of pods	Databases, uploads, anything that must survive
CSI drivers	Defined by the storage system	AWS EBS/EFS, Azure Disk/Files, GCE PD, Ceph, NFS

7.2 PersistentVolume and PersistentVolumeClaim

- **PersistentVolume (PV)** - a piece of real storage in the cluster, created by an admin or automatically. Cluster-scoped.
- **PersistentVolumeClaim (PVC)** - a request for storage by a user: "I need 1Gi, read-write by one node". Namespaced. Kubernetes **binds** the claim to a matching PV.
- **Pod** - refers only to the PVC by name. It never needs to know where the disk really is.

Everyday example
A gym has lockers (PVs) of different sizes. A member fills a request slip: "one medium locker" (PVC). The front desk assigns a locker that matches and gives the member the key. The member only remembers "my locker slip", not the locker's serial number. If the gym builds lockers on demand when a slip arrives, that is a StorageClass.

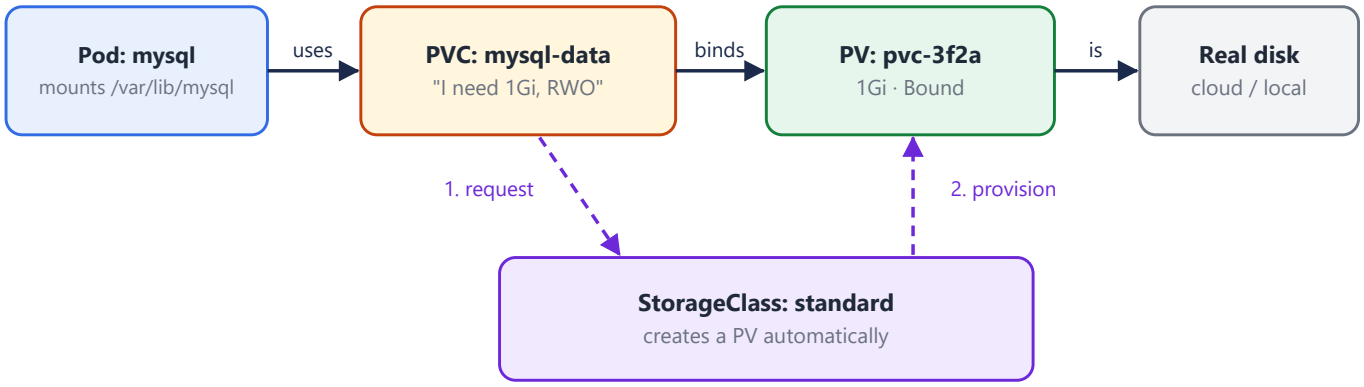


Figure 7.1 - A pod uses a PVC, which binds to a PV that a StorageClass can create on demand

pvc.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-data
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: standard        # minikube's default class
  resources:
    requests:
      storage: 1Gi
---
# in the pod spec
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: mysql-data
  containers:
    - name: mysql
      volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
```

Access mode	Short	Meaning
ReadWriteOnce	RWO	Mounted read-write by one node (most block disks)
ReadOnlyMany	ROX	Mounted read-only by many nodes

Access mode	Short	Meaning
ReadWriteMany	RWX	Read-write by many nodes (NFS, EFS, Azure Files, CephFS)
ReadWriteOncePod	RWOP	Read-write by exactly one pod

Reclaim policy	What happens to the PV when the PVC is deleted
Delete	The PV and the real disk are deleted (default for dynamic provisioning)
Retain	The PV and data are kept for manual recovery - use for important data
Recycle	Deprecated - basic scrub; do not use

7.3 StorageClass and dynamic provisioning

Creating PVs by hand does not scale. A **StorageClass** describes a type of storage (fast SSD, cheap HDD, replicated) and the **provisioner** that creates it. When a PVC names a StorageClass, the disk and PV are created automatically. One StorageClass can be marked as the cluster default.

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
provisioner: ebs.csi.aws.com           # on AWS; minikube uses k8s.io/minikube-hostpath
parameters:
  type: gp3
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer   # create the disk in the zone where the pod lands
```

Other features you should know: **volume expansion** (edit the PVC to a larger size if the class allows it) and **VolumeSnapshots** (point-in-time backups through CSI). Tools like **Velero** back up whole namespaces including volumes.

7.4 StatefulSets

A Deployment's pods are cattle: random names (`web-7d9f-x2b8l`), interchangeable, sharing one PVC if any. Databases, Kafka, Zookeeper and Elasticsearch need pets: each member has an identity and its own data. A **StatefulSet** provides:

- **Stable, ordered names** - `web-0`, `web-1`, `web-2` - kept after rescheduling.
- **Stable DNS** through a headless Service - `web-0.web-hl.default.svc.cluster.local`.
- **Per-pod storage** via `volumeClaimTemplates` - `data-web-0`, `data-web-1`... which follow the pod.
- **Ordered operations** - created 0, 1, 2 and deleted 2, 1, 0 (configurable with `podManagementPolicy`).
- **Rolling updates** from the highest ordinal down, with `partition` for staged rollouts.

> **Everyday example**
> Train seats are numbered. If passenger in seat 12 goes to the washroom and comes back, they return to seat 12 and their bag is still under it. That is a StatefulSet. A bus with free seating - sit anywhere, no fixed bag space - is a Deployment.

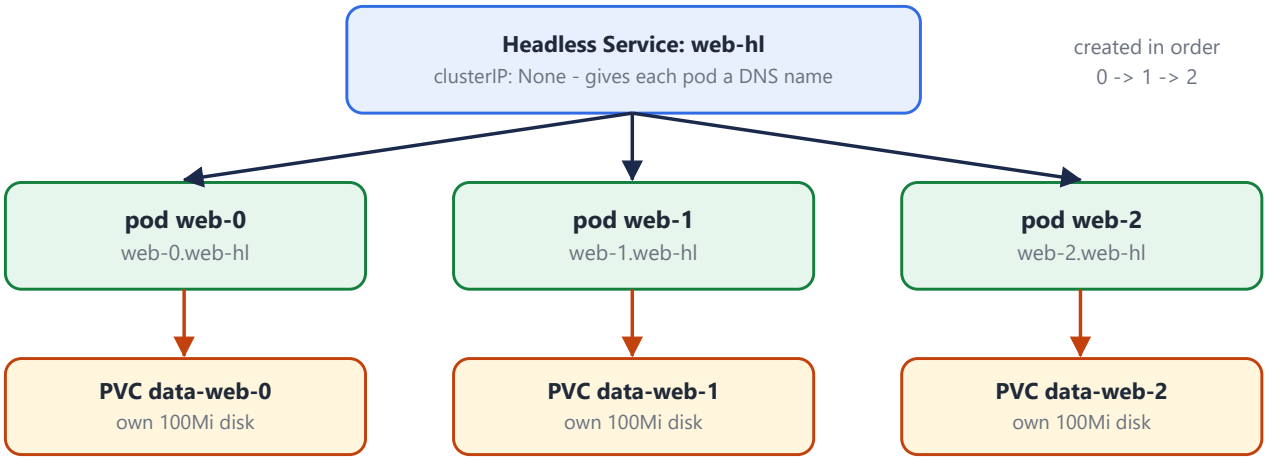


Figure 7.2 - StatefulSet pods keep their name, DNS entry and own PVC

statefulset.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: web-hl
spec:
  clusterIP: None              # headless
  selector: { app: web-sts }
  ports: [{ port: 80 }]
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: web-hl
  replicas: 3
  selector:
    matchLabels: { app: web-sts }
  template:
    metadata:
      labels: { app: web-sts }
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          volumeMounts:
            - name: data
              mountPath: /usr/share/nginx/html
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 100Mi
```

7.5 Live lab

PowerShell - part 1

```
kubectl get storageclass                      # 'standard (default)'
kubectl apply -f statefulset.yaml
kubectl get pods -l app=web-sts -w            # web-0, then web-1, then web-2
kubectl get pvc                               # data-web-0, data-web-1, data-web-2 - Bound
kubectl get pv

# give each pod its own page
0..2 | ForEach-Object {
  kubectl exec web-$_ -- sh -c "echo I am web-$_ > /usr/share/nginx/html/index.html"
}

kubectl run client --rm -it --image=busybox:1.36 -- sh
  wget -qO- web-1.web-hl                      # I am web-1
  nslookup web-hl                             # returns 3 pod IPs
  exit
```

PowerShell - part 2

```
kubectl delete pod web-1
kubectl get pods -l app=web-sts -w            # web-1 comes back with the SAME name
kubectl exec web-1 -- cat /usr/share/nginx/html/index.html   # still "I am web-1"

kubectl scale statefulset web --replicas=1    # web-2 then web-1 are removed
kubectl get pvc                               # PVCs are kept on purpose!
kubectl delete statefulset web; kubectl delete svc web-hl
kubectl delete pvc data-web-0 data-web-1 data-web-2
```

Common mistake

- Deleting a StatefulSet does **not** delete its PVCs - this protects data, but costs money in the cloud if forgotten.
- Running a database with RWO storage as a Deployment with 2+ replicas - the second pod cannot mount the disk or corrupts data.
- Thinking a StatefulSet makes a database highly available by itself - replication is the database's job; use an Operator for production databases.

Class 7 recap

- emptyDir lives with the pod; PVC data outlives pods.
- PV = the storage, PVC = the request, StorageClass = the factory.
- Access modes RWO/ROX/RWX/RWOP; reclaim Delete vs Retain.
- StatefulSet = stable names + stable DNS + per-pod PVC + ordering.

Class 7 - Top 20 interview questions

Try answering each question aloud before you read the model answer. Interviewers look for a clear definition, one real example, and one trade-off.

Q1. Why do we need volumes in Kubernetes?

A container's file system is ephemeral - data is lost when the container restarts or the pod is replaced. Volumes provide storage that survives restarts and can be shared between containers.

Q2. What is an emptyDir volume?

A volume created empty when a pod starts and deleted when the pod is removed. It survives container restarts and is used for scratch space or sharing files between containers. It can be RAM-backed with `medium: Memory`.

Q3. What is hostPath and why is it risky?

It mounts a file or directory from the node into the pod. It ties the pod to that node's data and can expose the host file system, so it is used only for system agents, not applications.

Q4. What is a PersistentVolume?

A cluster-scoped storage resource provisioned by an admin or dynamically by a StorageClass, with its own lifecycle independent of pods.

Q5. What is a PersistentVolumeClaim?

A namespaced request for storage specifying size, access mode and optionally a StorageClass. Kubernetes binds it to a suitable PV; pods use storage by referencing the PVC.

Q6. Explain the PV and PVC binding process.

When a PVC is created, the controller looks for a PV with enough capacity, matching access mode and class. If found, they bind one-to-one; if a StorageClass is set and no PV exists, one is dynamically provisioned. Otherwise the PVC stays Pending.

Q7. What is a StorageClass?

An object describing a class of storage - provisioner, parameters, reclaim policy, expansion and binding mode - used for dynamic provisioning of PVs.

Q8. What is dynamic provisioning?

Automatic creation of a PV and the underlying disk when a PVC requests a StorageClass, so admins do not need to pre-create volumes.

Q9. What access modes are available?

ReadWriteOnce (one node read-write), ReadOnlyMany (many nodes read-only), ReadWriteMany (many nodes read-write) and ReadWriteOncePod (exactly one pod read-write).

Q10. What are reclaim policies?

They decide what happens to a PV after its PVC is deleted: Delete removes the PV and backing storage, Retain keeps them for manual recovery, and Recycle is deprecated.

Q11. What is volumeBindingMode WaitForFirstConsumer?

It delays provisioning and binding until a pod using the PVC is scheduled, so the disk is created in the same zone as the pod. Immediate binding can place the disk in the wrong zone.

Q12. Can you resize a PVC?

Yes, if the StorageClass has `allowVolumeExpansion: true`. Edit the PVC to a larger size; shrinking is not supported.

Q13. What is CSI?

The Container Storage Interface - a standard that lets storage vendors write drivers for Kubernetes without changing Kubernetes code. EBS, Azure Disk and Ceph drivers are CSI drivers.

Q14. What is a StatefulSet?

A workload controller for stateful applications that gives each pod a stable ordinal name, stable network identity through a headless Service, its own persistent storage, and ordered deployment and scaling.

Q15. StatefulSet vs Deployment?

Deployment pods are interchangeable with random names and typically shared or no storage. StatefulSet pods have fixed names (app-0, app-1), stable DNS, per-pod PVCs and ordered start, stop and updates.

Q16. Why does a StatefulSet need a headless Service?

The headless Service creates DNS records for each pod, such as `web-0.web-hl.ns.svc.cluster.local`, so peers and clients can reach a specific member.

Q17. What are volumeClaimTemplates?

Templates in a StatefulSet from which a separate PVC is created for each pod (data-web-0, data-web-1). The PVC is reattached when that pod is rescheduled.

Q18. What happens to PVCs when you scale down or delete a StatefulSet?

By default they are kept to protect data. Newer versions allow `persistentVolumeClaimRetentionPolicy` to delete them on scale-down or deletion.

Q19. How would you run a production database on Kubernetes?

Use a StatefulSet managed by an Operator (for example CloudNativePG, Percona or Strimzi for Kafka) with replicated storage, backups, monitoring and anti-affinity - or use a managed cloud database.

Q20. What are VolumeSnapshots?

CSI-based point-in-time copies of a PVC, defined with VolumeSnapshot and VolumeSnapshotClass objects. A new PVC can be restored from a snapshot.